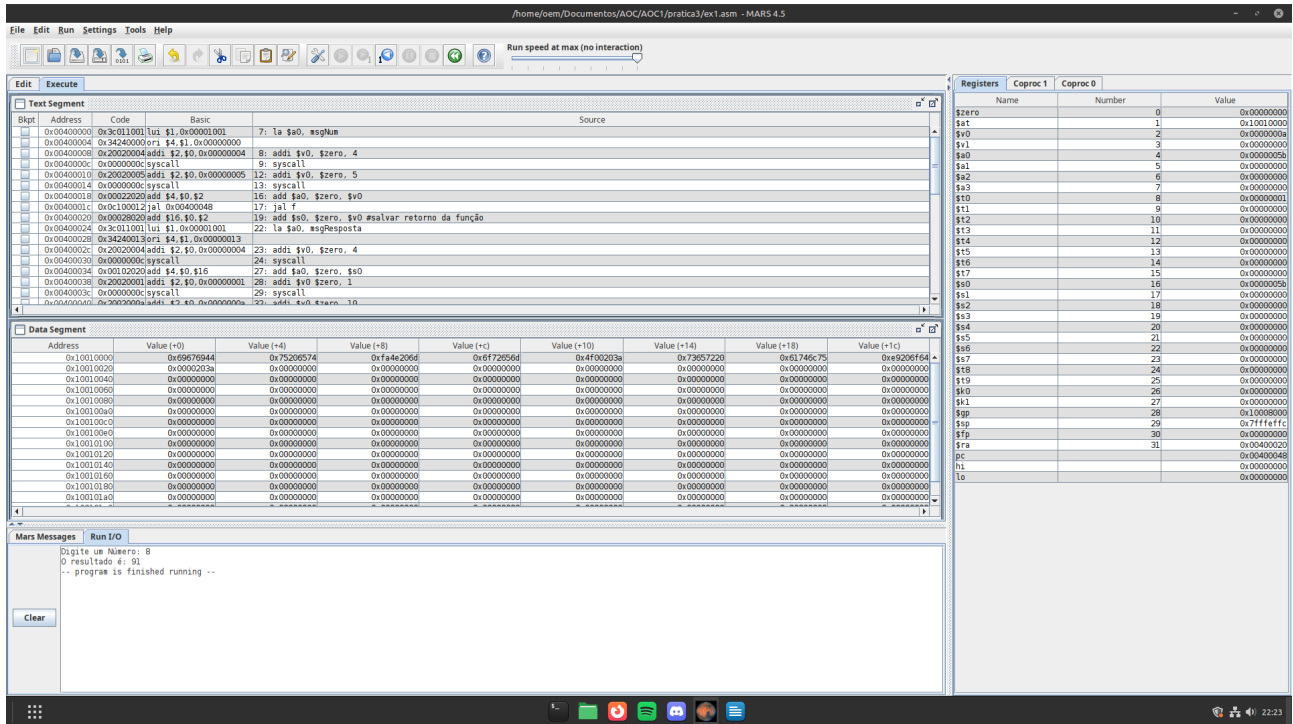


Relatório

Exercício 1



A principal decisão na implementação da função recursiva $H(N)$ foi o gerenciamento da pilha para salvar o endereço de retorno ($\$ra$), o argumento original N ($\$a0$) e os retornos parciais das duas chamadas recursivas usando os registradores $\$s0$ e $\$s1$. O caso base verifica se o número recebido é menor que 3, caso em que a função retorna o cálculo de $2N + 1$. A imagem de execução evidencia o correto funcionamento do programa no simulador MARS, recebendo a entrada 8 pelo console e calculando com precisão o resultado esperado de 91 para $H(8)$.

Exercício 2

The screenshot shows the MARS MIPS simulator interface. The main window displays the assembly code for Exercise 2. The Text Segment window shows instructions such as `7: la $a0, msgNum`, `8: addi $v0, $zero, 4`, `9: syscall`, and `10: jal f`. The Data Segment window shows memory addresses and their corresponding values. The Registers window shows the state of registers, with `$a0` containing 15. The Mars Messages window shows the program's output: `Digite um número: 15`, `O resultado é: 107`, and `-- program is finished running --`.

Na construção da função $F(N)$, a estrutura da pilha foi utilizada para armazenar apenas o argumento original `$a0` e o endereço de retorno `$ra`, já que o algoritmo exige apenas uma chamada recursiva ($F(N-4)$) em seu fluxo. O caso base foi tratado com a instrução `slt` para testar se $N < 4$; quando verdadeiro, o programa utiliza somas sucessivas do próprio registrador (`$a0 + $a0 + $a0`) para calcular a multiplicação por 3. A captura de tela comprova a funcionalidade correta, onde o programa recebe o valor 15 como entrada no prompt e resulta na saída 107 no console do MARS.

Exercício 3

The screenshot shows the MARS MIPS simulator interface. The main window displays the assembly code for Exercise 3. The Text Segment window shows instructions such as `9: la $s0, arr`, `10: addi $s1, $zero, 10 * tamanho`, `11: beq $t1, $zero, fimLoop`, and `12: sw $v0, 0($t1)`. The Data Segment window shows memory addresses and their corresponding values. The Registers window shows the state of registers, with `$a0` containing 10. The Mars Messages window shows the program's output: `Digite um número: 10`, `Digite um número: 5`, `Digite um número: 3`, `Digite um número: 7`, `Digite um número: 8`, `Digite um número: 27`, `Digite um número: 12345`, `Digite um número: 222`, `Digite um número: 225342`, `Digite um número: 425251`, and `O maior é: 425251`.

Na construção da função $F(N)$, a estrutura da pilha foi utilizada para armazenar apenas o argumento original `$a0` e o endereço de retorno `$ra`, já que o algoritmo exige apenas uma chamada recursiva ($F(N-4)$) em seu fluxo. O caso base foi tratado com a instrução `slt` para testar se $N < 4$; quando verdadeiro, o programa utiliza somas sucessivas do próprio

registrador (\$a0 + \$a0 + \$a0) para calcular a multiplicação por 3. A captura de tela comprova a funcionalidade correta, onde o programa recebe o valor 15 como entrada no prompt e resulta na saída 107 no console do MARS.

Exercício 4

The screenshot displays the MARS MIPS simulator interface. The main window is divided into several sections:

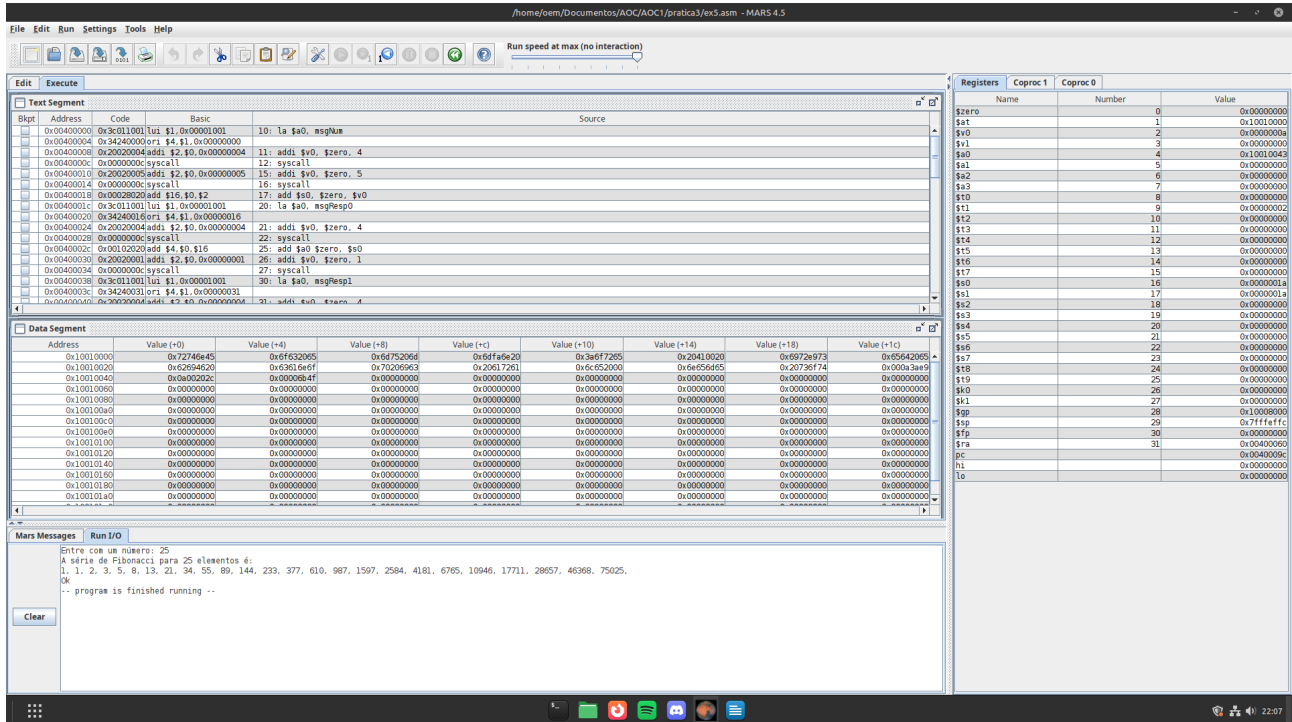
- Text Segment:** Shows assembly code with addresses, instructions, and comments. The code implements a recursive function to calculate the sum of numbers from 1 to 20.
- Registers:** A table showing the state of MIPS registers. The \$a0 register contains the value 15, which is the input to the program.
- Data Segment:** A table showing the state of memory locations. The values are all zero, indicating that the program has not yet written any data to memory.
- Mars Messages:** A console window showing the output of the program. The output is the sequence of numbers from 20 down to 1, printed in descending order.

The console output is as follows:

```
20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
-- program is finished running --
```

A implementação da função de impressão em ordem decrescente primeiramente verifica se o valor de entrada \$a0 é maior que zero para prosseguir, chamando recursivamente a mesma função ao subtrair 1 do parâmetro original. O programa foi projetado para executar as instruções de impressão (chamadas de syscall para número e espaço) antes da chamada recursiva, garantindo assim a apresentação sequencial do maior para o menor. A imagem do simulador exibe a execução para o valor de entrada 20, resultando na impressão correta da sequência contínua de "20 19 18 17..." até "1" na janela inferior.

Exercício 5



Para gerar a sequência de Fibonacci, o código adota uma abordagem de laço no método principal que avança a partir do elemento 1 até o número digitado pelo usuário, chamando a função recursiva de fibonacci em cada iteração para imprimir a série formatada com vírgulas e espaços. Na sub-rotina, o código testa os casos bases de $N=1$ e $N=2$ e faz amplo uso da pilha (reservando espaço para \$ra, \$s0 e \$a0) a fim de armazenar a soma dos retornos paralelos das chamadas de fibonacci($n-1$) e fibonacci($n-2$). A imagem do console MARS atesta a funcionalidade fornecendo o número 25 como entrada e mostrando a série correta de Fibonacci até seus 25 primeiros termos, finalizando perfeitamente no valor 75025